

Advanced Project

Foundations of Robotics

RUHAN SHAFI



UNIVERSITY OF
CANBERRA

Advanced Project

Introduction to Braitenberg Vehicles

Braitenberg vehicles are a class of simple autonomous agents conceived by the neuroscientist and cyberneticist Valentino Braitenberg in his 1984 thought experiment collection *Vehicles: Experiments in Synthetic Psychology* as simple thought experiments demonstrating how complex and seemingly intelligent behaviour can emerge from very simple sensor–motor connections. Braitenberg proposed a series of hypothetical “vehicles” equipped with basic sensors and motors, where the wiring between these components determined the robot’s behaviour [1]. His work became influential in cybernetics, artificial intelligence, behavioural robotics, and embodied cognition because it showed that sophisticated behaviour does not always require complex computation or programming.

Vehicle Type 1 is the simplest design. It contains a single sensor directly connected to a motor. In Vehicle 1, the robot responds proportionally to environmental stimuli such as light. For example, as light intensity increases, the motor speed may increase, causing the vehicle to move faster toward or away from the stimulus depending on the configuration. Although simple, this demonstrates reactive behaviour without decision-making algorithms.

Vehicle Type 2 expands upon this concept by using two sensors and two motors. Different wiring arrangements create dramatically different behaviours. In Vehicle 2a, ipsilateral connections are used, meaning each sensor controls the motor on the same side. This configuration typically causes the robot to avoid light sources because the motor nearest the light speeds up, turning the robot away. In Vehicle 2b, contralateral or crossed connections are used, where each sensor controls the opposite motor. This produces photophilic behaviour, causing the robot to move toward light sources in a lifelike and purposeful manner [2].

For the museum project, Vehicle 2b has been selected because its attraction to light creates more engaging and observable behaviour for visitors. The robot appears curious and responsive, making it more interactive for demonstrations. Additionally, Vehicle 2b better illustrates emergent behaviour principles central to modern autonomous robotics, swarm robotics, and behaviour-based control systems.

Task 2 - GL5528 Sensor & Vehicle 2b Motor Control Algorithm

Assumptions

1. The robot has two GL5528 LDR sensors in a voltage divider circuit; one mounted on the left side of the vehicle and one on the right side of the vehicle
2. The microcontroller reads each LDR as an analogue voltage value in the range 0–1023 (10-bit ADC)
3. Higher ADC reading = lower resistance = brighter light (as LDR resistance decreases with increasing illumination)
4. Two DC motors are controlled by a PWM signal in the range 0–255 (mapped from sensor range)
5. Vehicle 2b wiring: LEFT sensor drives RIGHT motor, RIGHT sensor drives LEFT motor (contralateral)
6. $MAX_ADC = 1023$, $MAX_PWM = 255$
7. A minimum speed threshold MIN_SPEED is applied so the vehicle always moves forward even in darkness
8. Motors accept a speed value between 0 and 255 via `SET_MOTOR.SPEED(motor, speed)`

```

1 // Braitenberg Vehicle 2b (Aggressor) - Light Seeking Behaviour
2 // Sensor:  GL5528 LDR in voltage divider (higher reading = more light)
3 // Wiring:  LEFT sensor → RIGHT motor | RIGHT sensor → LEFT motor
4 // Inputs:  Analogue ADC readings from left and right LDR sensors (0-1023)
5 // Outputs: PWM speed signals to left and right motors (0-255)
6
7 // Constants
8 MAX_ADC    ← 1023
9 MAX_PWM    ← 255
10 MIN_SPEED ← 50      // Minimum motor speed to ensure forward motion in darkness
11

```

```

12 METHOD BraitenbergVehicle2b()
13
14     WHILE (TRUE)
15
16         // Step 1: Read both light sensors
17         READ leftSensorValue      // ADC reading from left GL5528 (0-1023)
18         READ rightSensorValue     // ADC reading from right GL5528 (0-1023)
19
20         // Step 2: Map sensor readings to motor PWM speed values (0-255)
21         // Higher light reading on one side \rightarrow faster motor on opposite side
22         // This causes the vehicle to turn toward the brighter light source
23         leftMotorSpeed ← (rightSensorValue * MAX_PWM) / MAX_ADC
24         rightMotorSpeed ← (leftSensorValue * MAX_PWM) / MAX_ADC
25
26         // Step 3: Apply minimum speed threshold
27         // Ensures the vehicle keeps moving even when ambient light is very low
28         IF (leftMotorSpeed < MIN_SPEED) THEN
29             leftMotorSpeed ← MIN_SPEED
30         ENDIF
31
32         IF (rightMotorSpeed < MIN_SPEED) THEN
33             rightMotorSpeed ← MIN_SPEED
34         ENDIF
35
36         // Step 4: Clamp motor speeds to valid PWM range
37         IF (leftMotorSpeed > MAX_PWM) THEN
38             leftMotorSpeed ← MAX_PWM
39         ENDIF
40
41         IF (rightMotorSpeed > MAX_PWM) THEN
42             rightMotorSpeed ← MAX_PWM
43         ENDIF
44
45         // Step 5: Send computed speeds to motors
46         CALL SET_MOTOR_SPEED("LEFT", leftMotorSpeed)
47         CALL SET_MOTOR_SPEED("RIGHT", rightMotorSpeed)
48
49     ENDWHILE
50
51 ENDMETHOD

```

Task 3a - Distance Tracking: Sensor Selection and System Description

To track the total distance the Braitenberg vehicle travels in a day, the most appropriate and practical sensor choice is a Hall effect magnetic encoder attached to one of the drive wheels. A small permanent magnet is fixed to the wheel hub, and the Hall effect sensor is mounted on the chassis adjacent to it. Each time the wheel completes one rotation, the magnet passes the sensor and triggers a digital pulse. By counting these pulses over time and multiplying by the known circumference of the wheel, the total distance travelled can be continuously accumulated. The peripheral components required are the following: the Hall effect sensor itself (such as an A3144 or similar), a small neodymium magnet, a pull-up resistor on the signal line, and a microcontroller with a digital interrupt pin to count pulses reliably without polling.

Simplifying assumptions: the wheel does not slip on the museum floor surface; the wheel diameter is constant and known; both wheels travel the same distance (pure forward motion is assumed for distance summation purposes, meaning turning adds arc-length distance which is acceptable for a daily total); and the microcontroller remains powered for the full operating day with distance accumulated in a running total stored in memory, printed or displayed at the end of the day.

At the end of the museum's operating day, the total pulse count is multiplied by the wheel circumference ($\pi \times$ wheel diameter) divided by the number of magnets per revolution (one in this case) to yield the total distance in metres. This approach is mechanically simple, computationally low-cost, robust to the lighting conditions of the museum environment (unlike optical encoders which can be affected by ambient light, a relevant concern given the GL5528 sensors already in use), and entirely self-contained.

Task 3b - Distance Tracking Pseudocode

Assumptions

1. One Hall effect sensor is mounted on the left drive wheel
2. One magnet per wheel revolution produces one digital pulse per rotation
3. WHEEL_DIAMETER is known and fixed in metres
4. READ_ENCODER_PULSE() returns 1 if a new pulse has been detected since the last check, 0 otherwise
5. GET_TIME() returns current time in hours (24-hour format)
6. The operating day runs from START_HOUR (e.g. 9am) to END_HOUR (e.g. 5pm)

```
1 // Distance Tracking using Hall Effect Wheel Encoder
2 // Counts wheel rotations via magnetic pulses and converts to distance
3 // Resets at the start of each operating day and reports total at close
4
5 // Constants
6 WHEEL_DIAMETER ← 0.065           // Wheel diameter in metres (e.g. 65mm wheel)
7 WHEEL_CIRCUMFERENCE ← PI * WHEEL_DIAMETER // Distance per full rotation
8 PI ← 3.14159265
9 START_HOUR ← 9                   // Museum opening time (9:00am)
10 END_HOUR ← 17                   // Museum closing time (5:00pm)
11
12 METHOD TrackDailyDistance()
13
14     // Step 1: Initialise counters at start of day
15     pulseCount ← 0
16     totalDistance ← 0.0
17
18     PRINT "Distance tracking started."
19
20     // Step 2: Continuously count encoder pulses during operating hours
21     WHILE (GET_TIME() < END_HOUR)
22
23         // Read the encoder - returns 1 if a new pulse detected, else 0
24         encoderPulse ← READ_ENCODER_PULSE()
25
26         IF (encoderPulse = 1) THEN
27             // One pulse = one full wheel rotation
28             pulseCount ← pulseCount + 1
29             totalDistance ← pulseCount * WHEEL_CIRCUMFERENCE
30         ENDIF
31
32     ENDWHILE
33
34     // Step 3: Operating day has ended - report total distance
35     PRINT "Museum closed. Daily distance report:"
36     PRINT "Total rotations : " + pulseCount
37     PRINT "Total distance  : " + totalDistance + " metres"
38
39     RETURN totalDistance
40
41 ENDMETHOD
42
43 // --- Main Program ---
44 WHILE (GET_TIME() < START_HOUR)
45     // Wait until museum opens
46 ENDWHILE
47
48 CALL TrackDailyDistance() RETURNING totalDistance
```

References

- [1] Braitenberg, V. (1984). *Vehicles: Experiments in synthetic psychology*. MIT Press.
- [2] Wikipedia contributors. (2026, May 16). *Braitenberg vehicle*. Wikipedia. <https://wikipedia.org>